How Weather Affects Transportation

Team 5000: Akram Uddin, Abdul Muswara, Tanvir Islam, Hirwa Divin Iradukunda, Rawnak Ahmed, Diego Ordonez

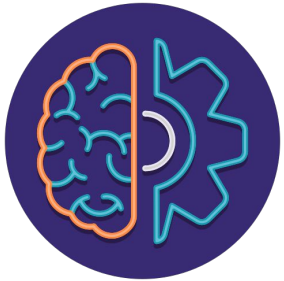
Problem Statement

We want to see…

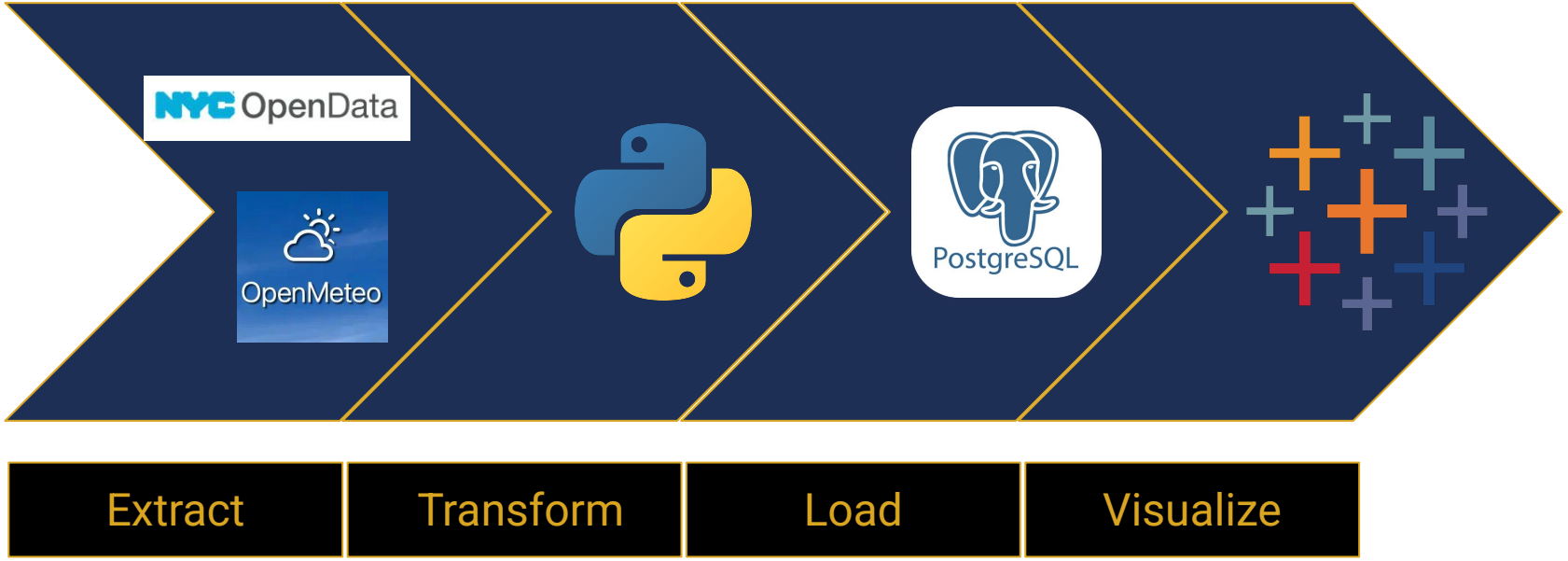
1. How MTA ridership and traffic volume changes depending on different weather conditions

2. What weather conditions have the greatest impact

3. If we can accurately predict these changes in the future with machine learning



Architecture



Datasets & APIs

Weather Source Open-Meteo API

Data Type: Temperature, rain, snowfall, wind speed

Bridges & Tunnels

Data Type: E-ZPass and toll vehicle counts

Traffic Volume

Data Type: Vehicle counts by street and direction

Bus Ridership

Data Type: Ridership and transfers by route

MTA (hourly) 2020–2024

Use Case: Identify shifts from driving to bus transit during poor weather

Subway Ridership

Data Type: Total station entries

MTA (hourly) 2020–2024

Use Case: Correlate subway demand spikes with rain and snow events

Seamless Collaboration with Docker

- **The Problem:** Coordinating environments across a 6-person team risks version conflicts and “it works on my machine” errors.
- **Solution:** Containerizing our PostgreSQL database and Python environment using Docker.
- **Why?:** Ensures the entire stack runs instantly on any machine with a single `docker-compose up` command, allowing us to focus on the data, not setup.

docker-compose.yml

```yaml
version: '3.8'

services:
  db:
    image: postgres:latest
    restart: always
    environment:
      POSTGRES_DB: mta_weather_db
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: password
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

  app:
    build: .
    volumes:
      - .:/app
    # This automatically loads our API key from the .env file
    env_file:
      - .env
    depends_on:
      - db

volumes:
  pgdata:
```

Why SQL?

- Our data is structured which SQL is excels at
- We have multiple datasets which we need to join together and SQL provides the tools and queries necessary to do this
- The schema of our data is defined and strict
- Cloud storage database management systems often use SQL such as Supabase which is something that fits our needs for this project.

PostgreSQL

- PostgreSQL as the 'brain' of our stack. While Supabase makes it easy for us to access our data.
- The Postgres engine underneath allows us to run the complex aggregations needed for certain models such as linear.
- Has the reliability needed to handle millions of rows provided by the MTA and NYC open data

The pivot to Supabase

- Supabase is an alternative to docker
- It’s a free web hosting platform which includes a fully managed PostgreSQL
- Easier to access and familiarize with
- Allows us move our project to our own server or to a different cloud provider in the future

Tableau for Visuals

- Tableau connects live to our PostgreSQL database (Supabase)

- Python writes master_hourly table for cleaned & merged data
- Stored in Supabase PostgreSQL

- Tableau connects via Native PostgreSQL connector
- Connect → To a Server → PostgreSQL

Live Connection

Pipeline run and Tableau visualization are connected end to end so the dashboard reflects whatever Python just wrote to the database

Tableau Dashboards

Weather vs Ridership/Traffic

A graph that shows the hourly ridership or traffic count based on the selected weather condition. Transportation method and weather condition will be chosen through parameters.

Forecast Traffic Prediction

A graph that shows the ridership or traffic count that a machine learning model predicts, based on upcoming forecast information.

NYC Traffic/Ridership Heatmap

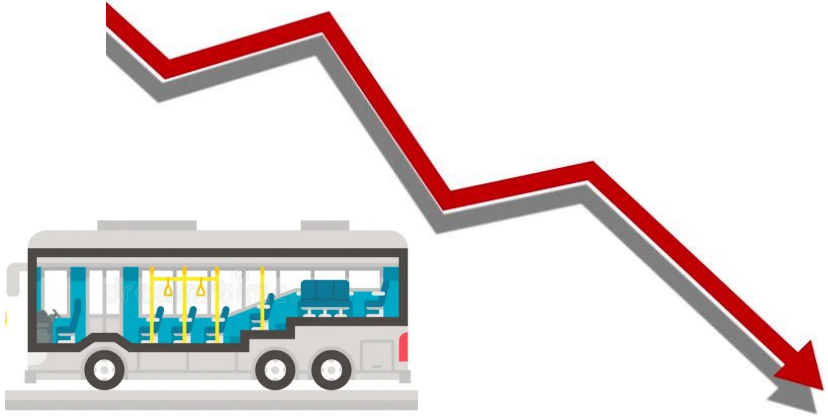
A map of NYC that shows the hotspots of traffic across the city to see where the ill effects of weather are most concentrated.

Models and Expected Insights

Linear Regression - Simple to interpret, by how much does MTA ridership and traffic volume change after say 1 inch of rain or snow

Logistic Regression - Given the forecast, is there a >70% chance of the MTA reporting high ridership.

Poisson Regression - 1 inch of rain reduces ridership by 15%



Team Roles

Akram Uddin (Data Researcher/Quality Assurance) Responsible for researching where to obtain data and necessary APIs. Makes sure the the data being used is trustworthy and high quality for the purpose of our problem	Abdul Muswara (Data Preprocessor/Data Scientist #1) Responsible for cleaning the data returned from APIs, and transforming it into an appropriate format for our SQL database.
Diego Ordonez (Data Preprocessor/Data Scientist #2) Responsible for cleaning the data returned from APIs, and transforming it into an appropriate format for our SQL database.	Tanvir Islam (Frontend Engineer/Tableau Visualizer #1) Responsible for connecting our SQL to Tableau and creating the dashboards and visualizations that aid in solving/answering our problem statement
Rawnak Ahmed (Frontend Engineer/Tableau Visualizer #2) Responsible for connecting our SQL to Tableau and creating the dashboards and visualizations that aid in solving/answering our problem statement	Hirwa Divin Iradukunda (Machine Learning Engineer) Responsible for choosing and fine tuning a machine learning model that will be able to accurately predict effects on transportation based on future forecasts

Ayon1113's Org FREE / NYC_Weather_VS_Traffic / main PRODUCTION Connect Feedback Search... ⌘K

Table Editor

schema public

New table

Search tables...

- bridges_tunnels_... UNRESTRICTE
- bus_hourly UNRESTRICTED
- master_hourly UNRESTRICTED
- subway_hourly UNRESTRICTED
- traffic_volume_h... UNRESTRICTED
- weather_h... UNRESTRICTED

subway_hourly | weather_hourly

Filter by timestamp, temperature_f, apparent_temp_f... or ask AI

Sort | RLS disabled | Index Advisor | Enable Realtime | Role postgres | Insert

timestamp timesta...	temperature_f float4	apparent_temp_f float4	precipitation_in float4	rain_in float4	snowfall_in float4	snow_depth_in float4	wind_speed_mph float4
2019-12-31 23:00:00	40.4735	31.7662	0	0	0	0	11.3648
2020-01-01 00:00:00	39.6635	30.0686	0	0	0	0	12.9456
2020-01-01 01:00:00	38.1335	28.6946	0	0	0	0	12.2075
2020-01-01 02:00:00	37.4135	28.5891	0	0	0	0	10.5305
2020-01-01 03:00:00	37.1435	27.4808	0	0	0	0	12.433

```python
def fetch_weather():
    print("Fetching weather data from Open-Meteo...")

    # Set up cached session with retry
    cache_session = requests_cache.CachedSession('.cache', expire_after=-1)
    retry_session = retry(cache_session, retries=5, backoff_factor=0.2)
    openmeteo = openmeteo_requests.Client(session=retry_session)

    params = {
        "latitude": 40.7143,
        "longitude": -74.006,
        "start_date": "2020-01-01",
        "end_date": "2024-12-31",
        "hourly": [
            "temperature_2m",
            "apparent_temperature",
            "precipitation",
            "rain",
            "snowfall",
            "snow_depth",
            "wind_speed_10m"
        ],
        "timezone": "America/New_York",
        "temperature_unit": "fahrenheit",
        "wind_speed_unit": "mph",
        "precipitation_unit": "inch"
    }

    responses = openmeteo.weather_api(
        "https://historical-forecast-api.open-meteo.com/v1/forecast",
        params=params
    )
    response = responses[0]
```

Thank You!

Any Questions?